📦 Python 元组 (Tuple) 核心学习笔记

🎯 1. 基础认知与操作

元组和列表非常相似，同样支持索引提取和 `for` 循环遍历。但它的核心特性是**不可变（只读）**，一旦创建，其内部元素的“指向”就不能再改变。

1.1 创建与读取

```
t = (1, 2, 3)
print(t)      # 输出整个元组：(1, 2, 3)
print(t[0])   # 通过索引提取第一个元素：1

# 遍历元组中的每个元素
for item in t:
    print(item)
```

1.2 严禁直接修改

因为元组是不可变的，如果尝试直接修改它的元素，Python 会无情报错（`TypeError`）：

```
# t[0] = 10  # ❌ 错误操作：元组不支持修改元素赋值
```

⚠️ 2. 核心避坑：单元素元组的“逗号法则”

当元组里只有一个元素时，**必须在元素后面加一个逗号 `,`**。否则，Python 会把小括号 `()` 当作普通的数学运算括号，而不是元组。

```
# ❌ 错误示范：这只是一个普通的字符串
tuple1 = ("xi")
print(type(tuple1))  # 结果是 <class 'str'>

# ☑ 正确示范：加上逗号，才是真正的元组
tuple2 = ("xi",)
print(type(tuple2))  # 结果是 <class 'tuple'>
```

🦉 3. 进阶魔法：元组中的“可变”元素

虽然元组本身是不可变的，但**如果元组里面嵌套了一个列表（可变对象），那么这个列表内部的元素是可以被修改的。**

这就像是一个上了锁的透明箱子（元组），里面装了一个没有上锁的小盒子（列表）。你不能把小盒子整个扔掉换成别的，但你可以伸手进去，改变小盒子里的东西。

3.1 代码实战演示

```
# 定义一个包含列表的元组
tuple3 = ("xi", "yao", [1, 2, 3])
print(tuple3)  # 输出: ('xi', 'yao', [1, 2, 3])

# 魔法 1: 修改嵌套列表里的值
tuple3[2][0] = 10
print(tuple3)  # 输出: ('xi', 'yao', [10, 2, 3])

# 魔法 2: 给嵌套列表追加新内容
tuple3[2].append("hello")
print(tuple3)  # 输出: ('xi', 'yao', [10, 2, 3, 'hello'])
```

3.2 为什么能这样改?

因为 `tuple3[2]` 存的只是那个列表的“内存地址”。只要这个地址没变（即你没有试图把 `[1,2,3]` 整个替换成别的字符串或数字），元组就不认为你破坏了它的“不可变”规则。